

1. Arbeidskrav

Prompten jeg brukte for å sammenligne de forskjellige HTML-kodene:

“copilot, gemini, uiogpt

Kan du sammenligne disse tre kode-filene og vurdere for meg med korte trekk hvilke løsning som er best?”

Deloppgave 2:

Gemini sin vurdering:

Løsning 2 er den beste.

Sammenligning av filene

Egenskap	Fil 1 (copilot)	Fil 2(gemini)	Fil 3(uiogpt)
CSS-struktur	Bruker inline <code>style=""</code> på elementene, noe som gjør koden uoversiktlig og vanskelig å vedlikeholde.	Samler all CSS i en egen <code><style></code> -blokk i <code><head></code> , noe som gir ryddig og skalerbar kode.	Samler CSS i en egen <code><style></code> -blokk i <code><head></code> .
Inndatavalidering	Sjekker ikke om feltet er tomt; gir utskrift selv uten inndata.	Inneholder validering (<code>.trim() === ""</code>) og gir gi beskjed dersom brukeren ikke har skrevet inn noe.	Sjekker ikke om feltet er tomt; gir utskrift selv uten inndata.

JS-syntaks	Bruker moderne JavaScript (const).	Bruker moderne JavaScript (const).	Bruker den eldre var -deklarasjonen i JavaScript.
Universell utforming (Accessibility)	Mangler <label> for tekstfeltet.	Mangler <label> for tekstfeltet.	Inneholder en formell <label for="ordInput"> .

Konklusjon og vurdering

- **1. plass – Løsning 2:** Dette er den mest robuste løsningen. Den har god struktur med sentrert design, bruker moderne JavaScript (**const**), og viktigst av alt håndterer den feil ved at den stopper tomme inputs.
- **2. plass – Løsning 3:** Har god CSS-struktur og er den eneste som inkluderer et **<label>**-element for tilgjengelighet. Den trekkes imidlertid ned av at den bruker utdatert JS-syntaks (**var**) og mangler sjekk for tomt tekstfelt.
- **3. plass – Løsning 1:** Fungerer rent teknisk, men inline-styling direkte på HTML-taggene er dårlig praksis og gjør koden vanskelig å drifte.

Fil 2 (kilde 2) er den definitivt beste løsningen.

Her er en kort sammenligning og vurdering av de tre kilde-filene:

- **Fil 1 ([copilot]):**
 - **Fordeler:** Veldig enkel og rett på sak.
 - **Ulemper:** Bruker inline CSS (**style="..."**), noe som gjør koden uoversiktlig og vanskelig å vedlikeholde. Den har ingen validering for tomme felt, noe som gjør at den skriver ut en rar melding dersom brukeren ikke har skrevet inn noe.

- **Fil 2 ([gemini]):**

- **Fordeler:** Bruker ryddig `<style>`-blokk i `<head>` med et moderne, sentrert kort-design. Den har god input-validering (`ord.trim() === ""`) som gir en vennlig feilmelding i stedet for feil resultat. Koden bruker moderne JavaScript (`const`).
- **Ulemper:** Ingen vesentlige ulemper for et enkelt skript.

- **Fil 3 ([uiogpt]):**

- **Fordeler:** Skiller også CSS ut i en `<style>`-blokk og har god tilgjengelighet ved å bruke en ekte `<label>` knyttet til input-feltet.
- **Ulemper:** Bruker utdatert JavaScript-syntaks (`var` i stedet for `const/let`) og streng-konkatinerer i stedet for mal-rekker. Mangler også validering for tomme strenger.

Konklusjon Fil 2(gemini) vinner fordi den kombinerer det mest visuelt gjennomførte designet, moderne JavaScript-praksis (`const` og mal-rekker), og har nødvendig logikk for å håndtere tomme inndata.

Copilot sin vurdering:

Her er en kort vurdering av de tre løsningene:

🏆 1. [gemini.html](#) (best) [\[gemini | HTML\]](#)

Styrker:

- Oppfyller alle kravene.
- Har pen og moderne styling.
- Håndterer tom input med en brukervennlig feilmelding.
- Strukturert kode med egne variabler for input- og resultatfelt.
- God lesbarhet og vedlikeholdbarhet.

Svakheter:

- Litt mer omfattende enn nødvendig for oppgaven.

Vurdering: Beste totale løsning fordi den både løser oppgaven korrekt og tar hensyn til brukeropplevelse. [\[gemini | HTML\]](#)

🥈 2. [uiogpt.html](#) [\[uiogpt | HTML\]](#)

Styrker:

- Oppfyller alle funksjonelle krav.
- Har ryddig layout og bruker `<label>`, som er bra for tilgjengelighet.
- Enkel og lett å forstå.

Svakheter:

- Ingen validering av tom input.
- Bruker `var` i stedet for moderne `const` eller `let`.
- Litt eldre JavaScript-stil med strengsammensetning ved hjelp av `+`.

Vurdering: Solid og ryddig løsning, men noe mindre moderne enn Gemini. [\[uiogpt | HTML\]](#)

🏆 3. [copilot.html](#) [\[copilot | HTML\]](#)

Styrker:

- Kort og presis.
- Bruker moderne JavaScript (`const` og template literals).
- Oppfyller oppgaven med svært lite kode.

Svakheter:

- Nesten all styling ligger som inline-attributter, noe som blir vanskeligere å vedlikeholde.
- Ingen håndtering av tom input.
- Mindre visuelt tiltalende enn de andre løsningene.

Vurdering: Teknisk korrekt, men enklest av de tre og minst robust. [\[copilot | HTML\]](#)

Samlet rangering

1. 🏆 [gemini.html](#) [\[gemini | HTML\]](#)
2. 🥈 [uiogpt.html](#) [\[uiogpt | HTML\]](#)
3. 🥉 [copilot.html](#) [\[copilot | HTML\]](#)

Hvis dette hadde vært en skoleoppgave eller en kodevurdering i arbeidssammenheng, ville jeg gitt **Gemini 9/10**, **UiOGPT 8/10** og **Copilot 7/10**.

UiO sin vurdering:

Her er en kort vurdering av de tre løsningene, fil for fil, basert på kodeutdragene du har gitt.

1. `uiogpt.html`

Kjennetegn

- Layout: Klassisk, enkel nettside med maks-bredde, sentrert innhold og nøytral bakgrunnsfarge, tilpasset lesbarhet på større skjermer
(CSS-styling av `body`, `h1`, `label`, `input`, `button`, `p`)
<ref: index=46996575 firstWord=1 lastWord=25/>
- Funksjon: Teller tegn i ordet og skriver ut tekst ala
`Ordet X består av Y tegn`
(JS som henter `ordInput`, bruker `ord.length` og bygger meldingen)
<ref: index=46996578 firstWord=20 lastWord=40/><ref: index=46996572 firstWord=1 lastWord=10/>

Fordeler

- Ren, lesbar kode og struktur.
- Klar og tydelig resultattekst.
- God, ryddig layout for oppgavens nivå.

Ulemper

- Ingen validering (f.eks. hvis feltet er tomt).
- Relativt "vanlig" styling – ikke like visuelt polert som Gemini-løsningen.

2. `gemini.html`

Kjennetegn

- Layout: Fullskjerms, sentrert kort (`.container`) med skygge, avrundede hjørner, moderne uttrykk
<ref: index=46996574 firstWord=1 lastWord=40/>
- Mer raffinert knapp-styling med hover-effekt
<ref: index=46996577 firstWord=10 lastWord=35/>
- Funksjon:
 - Leser ordet fra input
 - Sjekker om brukeren faktisk har skrevet noe (`ord.trim() === ""`)
 - Viser en feilmelding: `Vennligst skriv inn et ord først.`
 - Ellers: `Ordet 'X' består av Y tegn.`<ref: index=46996576 firstWord=10 lastWord=40/>

Fordeler

- Best brukeropplevelse:
 - Validering av tomt inputfelt.
 - Tydelig og hjelpsom feilmelding.

- Mest moderne og gjennomført visuelt design.
- God bruk av `const` og template literals (backticks).

****Ulemper****

- Litt mer kompleks CSS enn strengt nødvendig for en enkel oppgave (men fortsatt ryddig).

3. `copilot.html`

****Kjennetegn****

- Enkelt oppsett med tittel "Tell tegn i et ord" og knapp med f.eks. teksten "Sjekk"
 <ref: index=46996573 firstWord=1 lastWord=15/>
- Funksjon:
 - Leser `ordInput`
 - Teller tegn med `ord.length`
 - Viser: `Ordet X består av Y tegn`
 <ref: index=46996573 firstWord=20 lastWord=40/>

****Fordeler****

- Veldig enkel og lett å forstå.
- Gjør akkurat det oppgaven ber om, uten ekstra logikk.

****Ulemper****

- Ingen validering (tomt felt, mellomrom osv.).
- Mindre gjennomarbeidet layout enn de andre.
- Litt "rå" UX sammenlignet med de to andre.

Samlet vurdering: Hvilken er "best"?

1. **Best total-løsning: `gemini.html`**

På grunn av:

- Bedre UX (validering og feilmeldinger) <ref: index=46996576 firstWord=10 lastWord=30/>
- Mest gjennomført og moderne layout <ref: index=46996574 firstWord=1 lastWord=30/><ref: index=46996577 firstWord=10 lastWord=30/>

2. **Nummer to: `uiogpt.html`**

- God struktur, ryddig kode og grei visuell utforming <ref: index=46996575 firstWord=1 lastWord=25/><ref: index=46996578 firstWord=20 lastWord=40/>
- Mangler bare litt input-validering for å matche nivået til `gemini.html`.

3. ****Enklest (men svakest UX): `copilot.html`****

- Klarer funksjonskravet, men uten validering og med enklere visuell utforming <ref: index=46996573 firstWord=20 lastWord=40/>.

Hvis du vil, kan jeg foreslå en “kombinert” løsning som tar det beste fra alle tre (strukturen fra `uiogpt.html`, designet/valideringen fra `gemini.html`, og enkelheten fra `copilot.html`).

Deloppgave 3:

- **Er KI-modellene enige?**

KI-modellene er enige om at gemini har best filstruktur og mest moderne JS kode.

- **Hvilke KI-modell gir, etter din mening, det beste/riktige svaret?**

Jeg mener at gemini har best struktur og språk. Gemini og UiO sin KI-modell inneholder både html, css og js kode mens copilot kun har html og js kode.

- **Hvor godt er du i stand til å vurdere hvilken av KI-modellene som gir det beste / mest riktige svaret?**

Jeg har nok kunnskap i kode forståelse til å kunne se om koden inneholder riktig syntaks om html-koden er semantisk og hvordan css-koden er lagt opp. I uiogpt.filen så har de brukt variabelen var, som ikke lenger brukes. Den burde ikke brukes fordi verdiene kan endres akkurat som -let, men -var kan føre til uforutsigbare bugs/feil. Derfor mener jeg at jeg nok erfaring til å kunne sammenligne kode og velge ut i fra det.

- **Hvis du skulle løst denne oppgaven igjen, ville du promptet mer eller annerledes?**

Jeg hadde ikke promptet mer, men kanskje annerledes. I stedet for et ferdig kode sett, hadde jeg heller spurt KI-modellene om alternativer eller om hjelp for å forstå kode. Jeg hadde for eksempel hatt separat styling sheets med egen css-fil og ikke brukt <style> taggen.